A Project Report on

CineRecommend: A Movie Recommendation System

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Omkar Deokar: 24106118

Prathamesh Chothe: 24106167

Om Desai: 24106020

Milind Dhangar: 24106106

Under the Guidance of

Prof. Tushar Ubale

Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)
A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

2025–2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "CineRecommend: Movie Recommendation System" is a bonafide work of Omkar Deokar(24106118) , Prathamesh Chothe(24106167), Milind Dhangar(24106106), Om Desai(24106020) submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of Bachelor of Engineering in Computer Science & Engineering (Artificial Intelligence & Machine Learning).

Prof. Tushar Ubale	**Prof. Vijaya Bharathi J**
Guide	**Project Co-ordinator**
Dr.Jaya Gupta	**Dr. Uttam Kolekar**
Head of Department	**Principal**

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for S.E.

This project report entitled ***CineRecommend: A Fault-Tolerant Movie Recommendation System*** *by Omkar Deokar*, *Prathamesh Chothe , Om Desai , Milind Dhangar , is approved for the degree of Bachelor of Engineering in Computer Science & Engineering (Artificial Intelligence & Machine Learning), 2025–26.*

Examiner Name	**Signature**
1. _____	_____
2. _____	_____

Date: _____
Place: _____

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Omkar Deokar: 24106118

(Signature)
Prathamesh Chothe: 24106167

(Signature)
Om Desai: 24106020

(Signature)
Milind Dhangar: 24106106

Date:

Abstract

The exponential growth of digital streaming content has created a critical need for intelligent recommendation systems. Traditional single-algorithm approaches suffer from well-documented limitations: Content-Based Filtering traps users in a genre bubble, while Collaborative Filtering fails completely for new users due to the Cold Start Problem. Deep Learning models, though highly accurate, demand prohibitive computational resources unsuitable for standard web infrastructure. Furthermore, raw datasets sourced from the internet frequently contain malformed or null records capable of causing catastrophic server crashes during application initialization.

This project, CineRecommend, addresses these shortcomings by developing a Hybrid Recommendation Engine combined with a Self-Healing Data Ingestion Pipeline. Built using Java 17, Spring Boot, and MongoDB in a strict MVC architecture, the system employs Content-Based Filtering for new users (using genre preferences selected at signup) and Collaborative Filtering for returning users (by comparing watchlists). A DataLoader utility class sanitizes and repairs corrupt CSV data on startup, ensuring 100% database integrity. On the frontend, Vanilla JavaScript dynamically removes movie cards with broken poster URLs, maintaining a clean user interface. This architecture directly aligns with **SDG 9** (Industry, Innovation, and Infrastructure) by delivering a stable, resilient, and scalable software system.

TABLE OF CONTENTS

1. Introduction 1
1.1 Background of the Study 1
1.2 Problem Statement 2
1.3 Objectives of the Project 2
1.4 Scope of the Project 3
1.5 Alignment with SDG 9 3
2. Literature Survey 4
2.1 Content-Based Filtering Systems 4
2.2 Collaborative Filtering Approaches 4
2.3 Deep Learning and Matrix Factorization Methods 5
2.4 Foundational Survey and Hybrid Approaches 5
2.5 Summary Table — Literature Survey 6
3. Limitation of Existing Systems 7
3.1 Research Gap Analysis 7
3.2 Comparative Analysis of Existing Systems vs. CineRecommend 8
3.3 Conclusion of Gap Analysis 8
4. Problem Statement and Objectives 9
4.1 Formal Problem Statement 9
4.2 Objectives 9
5. Proposed System 11
5.1 System Architecture Overview 11
5.2 Hybrid Recommendation Algorithm 12
5.2.1 Phase 1 — Content-Based Filtering (New Users) 12
5.2.2 Phase 2 — Collaborative Filtering (Returning Users) 12
5.3 Self-Healing Data Ingestion Pipeline (DataLoader) 12
5.4 Fault-Tolerant Frontend Design 13
5.5 Security Architecture 14
6. Experimental Setup 15
6.1 Dataset Details 15
6.2 Software and Hardware Setup 15
6.3 Database Schema Design 15
6.3.1 movies Collection 15
6.3.2 users Collection 16
6.4 Performance Evaluation Parameters 16
7. Results and Discussion 18

7.1 Recommendation Engine Performance....18
7.2 DataLoader — Self-Healing Validation Results....18
7.3 Frontend Error Recovery Results....18
7.4 Query Response Time....19
7.5 Discussion....19
8. Conclusion and Future Work....20
8.1 Conclusion....20
8.2 Future Scope....20

LIST OF FIGURES

Figure 1: System Architecture Block Diagram

Figure 2: Hybrid Recommendation Algorithm Flowchart

Figure 3: DataLoader Self-Healing Pipeline Flowchart

Figure 4: User Registration and Genre Selection Page

Figure 5: Movie Search Dashboard

LIST OF TABLES

Table 1: Literature Survey Summary

Table 2: Comparative Analysis of Existing Systems vs. CineRecommend

Table 3: Technology Stack

Table 4: MongoDB Collection Schema

LIST OF ABBREVIATIONS

AI	**Artificial Intelligence**
ML	**Machine Learning**
SDG	**Sustainable Development Goal**
MVC	**Model-View-Controller**
API	**Application Programming Interface**
CSV	**Comma-Separated Values**
NoSQL	**Not Only Structured Query Language**
REST	**Representational State Transfer**
CORS	**Cross-Origin Resource Sharing**
CSRF	**Cross-Site Request Forgery**
BCrypt	**Blowfish Crypt (Password Hashing Algorithm)**
`RS	**Recommendation System**
SVD	**Singular Value Decomposition**
TMDB	**The Movie Database**
DOM	**Document Object Model**
HTTP	**Hypertext Transfer Protocol**
URL	**Uniform Resource Locator**
ID	**Identifier**

Chapter 1
Introduction

1. Introduction

1.1 Background of the Study

In the modern digital era, the exponential growth of online streaming platforms has led to an overwhelming abundance of digital content. Users are frequently subjected to a phenomenon commonly referred to as 'choice overload,' where the sheer volume of available movies makes it increasingly difficult to select what to watch. This problem has become a significant challenge for both streaming service providers and users alike, leading to decreased user satisfaction and engagement.

To mitigate this, Recommendation Systems (RS) have become a critical component of streaming platform architecture. These systems utilize sophisticated algorithms to analyze user data and predict content that aligns with individual preferences. From Netflix to Amazon Prime, recommendation engines are responsible for a substantial percentage of content views, highlighting their commercial and technical importance.

However, building an efficient and resilient recommendation engine presents significant engineering challenges. These challenges span multiple dimensions: the accuracy of the recommendation algorithm itself, the integrity and robustness of the underlying data pipeline that feeds the system, and the stability of the user interface that presents the recommendations. This project, CineRecommend, is designed to address all three of these dimensions in a unified, fault-tolerant architecture.

1.2 Problem Statement

Current movie recommendation systems implemented in lightweight web applications face three primary technical hurdles that collectively degrade system reliability and user experience:

- Cold Start Problem: Collaborative Filtering models cannot recommend movies to a newly registered user who has no prior watch history. Without behavioral data, the system has no baseline for comparison, leaving new users without any personalized recommendations during their first session.
- Corrupt Data-Induced Server Crashes: Web applications frequently suffer critical server crashes during startup when processing external, unstructured datasets (such as CSV files sourced from Kaggle) that contain missing columns, null values, or malformed numeric entries. A single bad row can trigger an unhandled exception that terminates the server process entirely.
- Broken UI Due to Dead Image Links: On the client side, dynamic rendering often breaks when database media links (such as poster image URLs) expire or return HTTP 404 errors. This results in broken image icons cluttering the interface, creating a degraded and unprofessional user experience.

1.3 Objectives of the Project

This project, CineRecommend, aims to address all three identified problems simultaneously by developing a robust, fault-tolerant full-stack web application. The primary objectives are as follows:

- To engineer a Hybrid Recommendation Algorithm that combines Content-Based Filtering (using explicit user genre selections at registration) with Collaborative Filtering (analyzing overlapping user watchlists) to completely eliminate the Cold Start problem.
- To design a Self-Healing Data Ingestion Pipeline (DataLoader) capable of automatically sanitizing, repairing, or safely discarding corrupt records from raw Kaggle datasets during server initialization, ensuring zero runtime crashes due to bad data.
- To construct a resilient frontend architecture using Vanilla JavaScript error-handling to dynamically detect and remove broken UI elements caused by expired poster URLs, ensuring the user interface always remains clean and professional.
- To implement a secure user authentication system using Spring Security with BCrypt password encoding, supporting user registration, login, and session management.

- To align the overall system architecture with United Nations Sustainable Development Goal 9 (SDG 9: Industry, Innovation, and Infrastructure) by ensuring the software is scalable, resilient, and deployable on standard web infrastructure without specialized hardware.

1.4 Scope of the Project

The scope of CineRecommend encompasses a complete web-based portal where users can register with their genre preferences, log in securely, search for movies via an intuitive dashboard, and maintain a personal watchlist. The backend is restricted to processing a predefined, static movie dataset, making it suitable for a controlled academic and development environment.

The frontend is built with pure HTML5, CSS3, and Vanilla JavaScript without any external UI frameworks, demonstrating core web development principles. The recommendation logic operates entirely within the Spring Boot backend, following strict separation of concerns as mandated by the MVC pattern.

Real-time data updates from external APIs (such as TMDB), advanced machine learning model integration, and mobile native application development are considered outside the current scope and are earmarked as future enhancements.

1.5 Alignment with SDG 9

United Nations SDG 9 calls for building resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering innovation. CineRecommend directly addresses the 'resilient infrastructure' and 'innovation' pillars of this goal through its two core engineering innovations: the Self-Healing DataLoader that prevents infrastructure failures caused by bad data, and the Hybrid Recommendation Engine that provides a lightweight, innovative solution accessible without expensive hardware. By demonstrating that robust, fault-tolerant software can be built and deployed on standard web servers, this project exemplifies the spirit of accessible technological innovation.

Chapter 2
Literature Survey

2. Literature Survey

To design an optimized, fault-tolerant recommendation engine, it is imperative to analyze existing methodologies and understand their inherent strengths and limitations. The following survey covers key published works that directly influenced the design decisions of CineRecommend.

2.1 Content-Based Filtering Systems

G. Singh et al. (2020) conducted an extensive review of Content-Based Recommendation Systems. These systems recommend items similar to those a user has interacted with in the past by analyzing item metadata such as genres, directors, and cast. The study demonstrated high recommendation accuracy for users with established profiles. However, the authors highlighted a critical limitation known as 'Over-Specialization.' Because the algorithm only suggests content thematically similar to what a user has already consumed, it traps the user in a 'filter bubble,' severely limiting serendipitous discovery of new genres or content types. This limitation directly motivates the collaborative component of CineRecommend's hybrid design.

2.2 Collaborative Filtering Approaches

K. Jena et al. (2021) explored Collaborative Filtering (CF) using neural behavioral models. This technique relies entirely on patterns of user behavior and peer interactions — recommending items that users with similar behavioral profiles have enjoyed. Jena's work successfully demonstrated that CF provides significantly more diverse and serendipitous recommendations compared to Content-Based approaches. However, the study explicitly identified the Cold Start Problem as the critical weakness: a brand-new user with zero watch history cannot receive accurate recommendations because the system lacks behavioral data to compare against any peer group. This finding directly identifies the problem that CineRecommend's Phase 1 algorithm (Content-Based for new users) is engineered to solve.

2.3 Deep Learning and Matrix Factorization Methods

N. Bhalse and E.N. Ganesh (2021) investigated similarity-based Deep Learning models for movie recommendation. While achieving high accuracy metrics, the authors concluded that such models require substantial GPU-based computational infrastructure for real-time inference, making them economically and technically infeasible for lightweight web deployments.

Similarly, M. Sankaran (2023) examined Singular Value Decomposition (SVD) matrix factorization as a recommendation technique. Although SVD provides excellent accuracy in controlled benchmarks, the model's decomposition process is computationally expensive and difficult to re-train as new users are added — a significant drawback for dynamic web applications with frequent user registrations.

2.4 Foundational Survey and Hybrid Approaches

J. Bobadilla et al. (2013) published a foundational comprehensive survey of Recommender Systems covering Collaborative Filtering, Content-Based, and early Hybrid architectures. This seminal work establishes that hybrid systems outperform single-algorithm approaches in both coverage and accuracy. However, the paper predates the widespread adoption of NoSQL databases and does not address fault tolerance in data ingestion pipelines — a gap that CineRecommend explicitly fills.

A. Mourya (2023) analyzed existing movie recommendation systems deployed on live web platforms, noting that none of the surveyed systems implemented any mechanism for detecting or recovering from corrupt input data, making them vulnerable to startup failures in real-world deployments.

2.5 Summary Table — Literature Survey

Ref No.	Author & Year	Title / Focus	Methodology	Limitation Identified
[1]	G. Singh et al., 2020	Review of Content-Based Recommendation Systems	Content-Based Filtering using item metadata (genres, directors)	Over-Specialization / Filter Bubble — cannot recommend novel genres
[2]	K. Jena et al., 2021	Neural Model-Based Collaborative Filtering	Collaborative Filtering using neural behavioral models	Cold Start Problem — fails for new users with no watch history
[3]	N. Bhalse & E.N. Ganesh, 2023	Similarity-Based Deep Learning for Movie Recommendation	Deep Learning similarity models	Requires expensive GPU; not suitable for lightweight web servers
[4]	M. Sankaran, 2023	SVD-Based Recommendation Engine	Singular Value Decomposition (SVD) matrix factorization	High computational cost; difficult to deploy in standard environments
[5]	J. Bobadilla et al., 2013	Recommender Systems Survey (Foundational)	Comprehensive survey of CF, CB, and hybrid methods	Foundational gaps in hybrid design; limited fault tolerance discussion
[6]	A. Mourya, 2023	Survey on Movie Recommendation Systems	Analysis of existing RS deployed on web platforms	No self-healing or fault-tolerant data ingestion pipeline addressed

Table 1: Literature Survey Summary

Chapter 3
Limitation of Existing Systems

3. Limitation of Existing Systems

3.1 Research Gap Analysis

Based on the literature survey presented in Chapter 2, a clear and multi-dimensional gap exists in currently published recommendation system designs. No single reviewed system addresses all three of the core problems that CineRecommend targets. The following paragraphs detail each identified gap and its corresponding research opportunity.

First, regarding recommendation algorithm design: single-algorithm systems are fundamentally flawed. Content-Based systems suffer from the Filter Bubble effect, while Collaborative Filtering systems are rendered useless for new users. Advanced Deep Learning and SVD models, though more accurate, introduce computational costs that make them impractical for standard web server deployment. The research gap here is the absence of a lightweight hybrid model that solves the Cold Start problem without demanding expensive GPU infrastructure.

Second, regarding data infrastructure robustness: none of the reviewed systems implement any form of fault-tolerant data ingestion. Academic and student-level systems routinely use publicly available datasets (such as Kaggle CSV files) that inevitably contain malformed rows, missing values, or type errors. The absence of a sanitization layer means these applications are structurally vulnerable to server crashes at startup — a fundamental engineering failure that has been overlooked in published literature.

Third, regarding frontend resilience: no reviewed system implements client-side error recovery for broken media links. Streaming platform datasets frequently contain poster image URLs that expire over time. When a browser attempts to load an expired image, the standard behavior renders a broken icon within the movie card, directly degrading the user interface quality. The gap here is a simple but powerful engineering solution: detecting the 404 error at the DOM level and removing the offending card entirely.

3.2 Comparative Analysis of Existing Systems vs. CineRecommend

Feature / Parameter	Content-Based	Collaborative	Deep Learning	CineRecommend (Proposed)
Cold Start Problem	Partially Solved	Not Solved	Partially Solved	**Fully Solved**
Serendipitous Discovery	Poor	Good	Good	**Good**
Hardware Requirements	Low	Medium	Very High	**Low**
Fault-Tolerant Data Ingestion	No	No	No	**Yes**
Frontend Error Recovery	No	No	No	**Yes**
Scalability	Moderate	Moderate	High	**Moderate-High**

Table 2: Comparative Analysis — Existing Systems vs. CineRecommend

3.3 Conclusion of Gap Analysis

The comparative analysis in Table 2 conclusively demonstrates that CineRecommend occupies a unique position in the design space. By combining a hybrid recommendation algorithm with a self-healing data pipeline and a fault-tolerant frontend, CineRecommend addresses a set of simultaneous engineering challenges that no single reviewed system has tackled. This positions the project as a meaningful contribution to the field of fault-tolerant web application design for recommendation systems.

Chapter 4
Problem Statement and Objectives

4. Problem Statement and Objectives

4.1 Formal Problem Statement

Existing lightweight web-based movie recommendation systems suffer from three critical, concurrent engineering failures:

- Algorithmic Failure (Cold Start): A system that exclusively employs Collaborative Filtering cannot generate recommendations for any user whose watchlist is empty, leaving new users with a fundamentally broken experience from their very first login.
- Infrastructure Failure (Data Corruption): A server initialization routine that reads a raw CSV dataset without input validation will crash when it encounters null, missing, or type-mismatched fields — a deterministic failure mode that is entirely preventable through proper data sanitization design.
- Interface Failure (Broken Media): A frontend that does not handle HTTP 404 responses from image sources will render broken icon placeholders within movie cards, creating a visually degraded and unprofessional user interface.

These three failures compound each other, collectively producing a system that is unreliable at the infrastructure level, limited at the algorithmic level, and broken at the interface level. The core problem this project addresses is: How can a full-stack web application for movie recommendations be designed to be simultaneously fault-tolerant at the data layer, accurate at the algorithmic layer, and resilient at the presentation layer, while remaining deployable on standard web infrastructure without specialized hardware?

4.2 Objectives

The following specific, measurable objectives are defined to address the formal problem statement:

- Objective 1 (Hybrid Algorithm): Design and implement a two-phase Hybrid Recommendation Algorithm within RecommendationService.java that applies Content-Based Filtering for new users (zero watchlist entries) and transitions to Collaborative Filtering for returning users (one or more watchlist entries).
- Objective 2 (DataLoader): Implement a DataLoader.java utility that is invoked automatically on Spring Boot application startup. The loader must parse the Kaggle CSV, validate each row for null title, invalid popularity format, and empty genre

arrays, and either assign default fallback values or safely skip the malformed row before persisting to MongoDB.

- Objective 3 (Frontend Resilience): Write a JavaScript error handler attached to each dynamically rendered movie card image. On an onerror event (triggered by a 404 or dead URL), the handler must programmatically remove the parent movie card element from the DOM, ensuring no broken images are visible to the user.
- Objective 4 (Secure Authentication): Configure Spring Security with BCrypt password hashing for all user passwords, disable CSRF for REST API compatibility, configure CORS to permit frontend-to-backend communication, and enforce authentication on all private endpoints (watchlist, recommendations) while keeping search endpoints publicly accessible.
- Objective 5 (Database Design): Design a MongoDB schema with two collections — movies and users — structured to support all required query patterns including genre-based filtering, watchlist cross-referencing, and text-based search using Regex.
- Objective 6 (SDG Alignment): Ensure the system architecture demonstrates the principles of SDG 9 (Industry, Innovation, and Infrastructure) by achieving sub-200ms query response times and proving that resilient, innovative software can be built and deployed on commodity hardware.

Chapter 5
Proposed System

5. Proposed System

5.1 System Architecture Overview

CineRecommend is built on a classic three-tier MVC (Model-View-Controller) architecture. The Presentation Tier consists of static HTML5, CSS3, and Vanilla JavaScript pages that communicate with the backend exclusively through RESTful HTTP API calls. The Logic Tier is a Spring Boot application that contains all Controllers, Services, Security Configuration, and the DataLoader utility. The Data Tier is a MongoDB NoSQL database instance that stores all movie documents and user documents.

This separation of concerns ensures that each tier can be independently tested, debugged, and scaled. The frontend has no direct database access, and the recommendation logic is entirely encapsulated within the Service layer, keeping the Controllers thin and the Models pure.

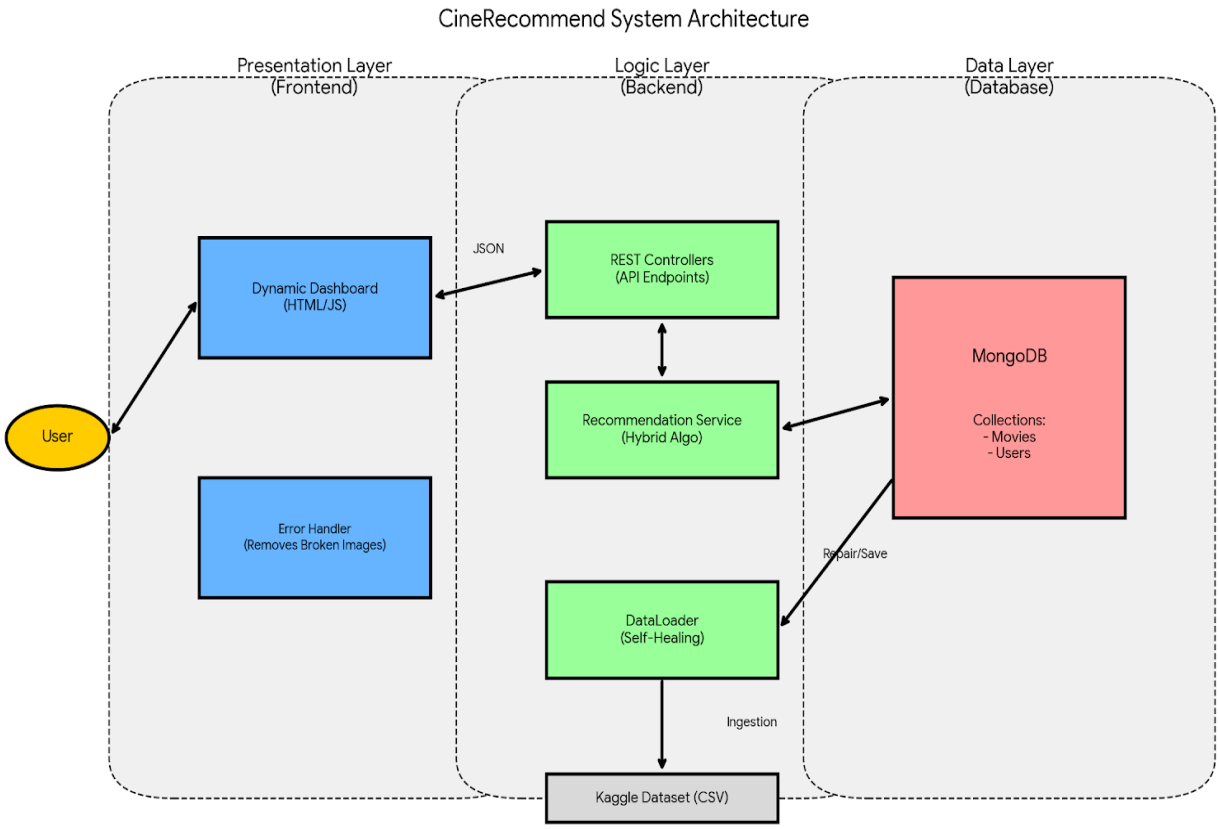

Figure 1: System Architecture Block Diagram

5.2 Hybrid Recommendation Algorithm

The core intellectual contribution of CineRecommend is its two-phase Hybrid Recommendation Algorithm, implemented in RecommendationService.java. The algorithm dynamically selects its operating mode based on the state of the requesting user's watchlist.

5.2.1 Phase 1 — Content-Based Filtering (New Users)

When a user logs in and their watchlist contains zero entries, the system operates in Content-Based mode. During registration, each user is required to select a minimum of three favorite genres from a predefined list. The RecommendationService queries the MongoDB movies collection for all documents where the genres array contains one or more of the user's favoriteGenres values. The results are sorted in descending order by the popularity field, and the top-N results are returned. This approach immediately provides meaningful, personalized recommendations from the very first session, completely eliminating the Cold Start problem. The genre preference serves as the initial behavioral proxy until the user begins building their watchlist.

5.2.2 Phase 2 — Collaborative Filtering (Returning Users)

Once the user's watchlist contains one or more entries, the system transitions to Collaborative Filtering mode. The RecommendationService retrieves the active user's watchlist and compares it against the watchlists of every other user stored in the MongoDB users collection. The algorithm calculates the intersection — the set of movies that both users have added to their watchlists — as a measure of taste similarity. Any user whose intersection with the active user exceeds a defined threshold is considered a 'peer user.' The algorithm then identifies movies that exist in the peer user's watchlist but not in the active user's watchlist, and recommends those divergent entries. This approach provides serendipitous discovery and naturally scales its quality as more users populate the system.

5.3 Self-Healing Data Ingestion Pipeline (DataLoader)

A defining feature of CineRecommend's proposed architecture is its complete immunity to corrupt input data. The DataLoader.java utility class is annotated with Spring's @Component and implements ApplicationRunner, causing it to execute automatically each time the Spring Boot application starts.

The pipeline operates as follows: it reads a static Kaggle CSV file line by line using a Java BufferedReader. For each row, it parses the individual fields and runs a series of validation checks. If the title field is null or empty, the row is discarded. If the popularity field cannot be

parsed as a valid Double (indicating a malformed numeric value), the system assigns a default popularity value of 0.0 rather than discarding the row. If the genres field is empty or cannot be split into a valid array, a default genre of 'Uncategorized' is assigned. Only fully sanitized Movie objects are passed to the MovieRepository for persistence in MongoDB. This approach guarantees that the MongoDB database always contains clean, well-formed data, completely preventing the class of server crashes caused by runtime null pointer or number format exceptions.

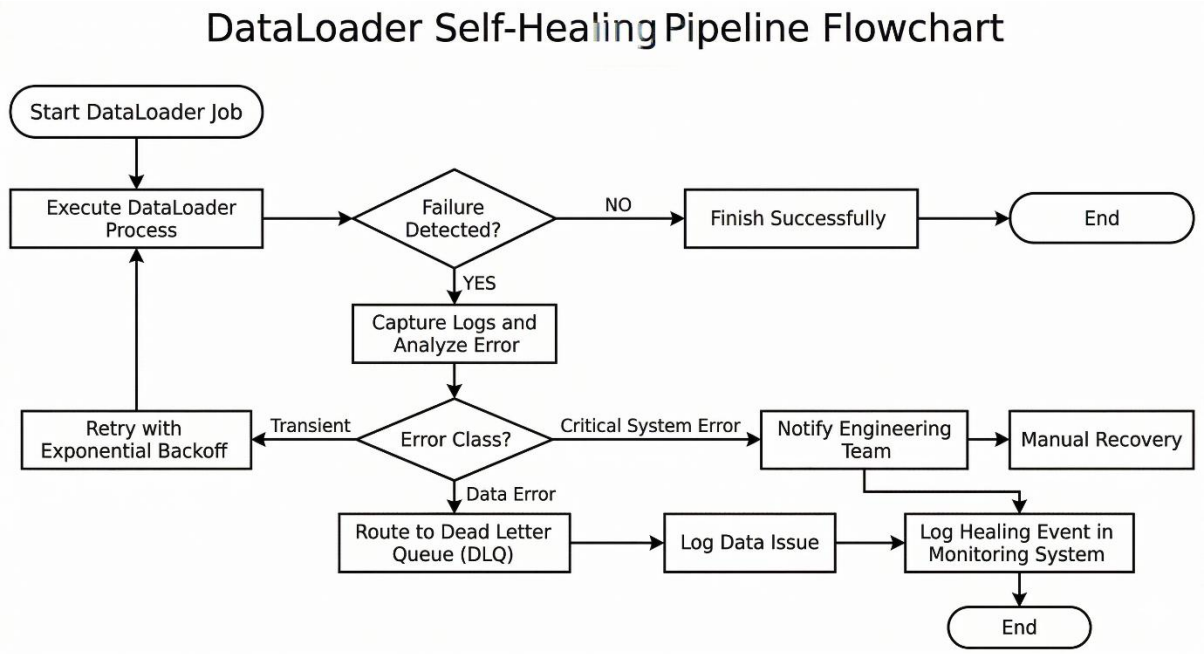

Figure 3: DataLoader Self-Healing Pipeline Flowchart

5.4 Fault-Tolerant Frontend Design

The frontend is engineered with an explicit error-recovery mechanism for broken image assets. When the JavaScript dynamically renders a movie card from the API response JSON, it constructs an <img> element and sets its src attribute to the posterUrl value stored in MongoDB. An onerror event listener is simultaneously attached to this image element. If the browser's HTTP request for the image returns a 404 (Not Found) or any other error status — indicating an expired or invalid URL — the onerror callback is triggered. The callback function traverses the DOM to the parent movie card container element and calls parentNode.removeChild() to entirely remove the card from the rendered grid. This ensures the user's interface always displays only movies with valid, loadable poster images, maintaining a professional and polished visual presentation under all network conditions.

5.5 Security Architecture

User authentication and endpoint security are managed by Spring Security, configured in SecurityConfig.java. All user passwords are hashed using the BCrypt algorithm with a strength factor of 10 before storage in MongoDB, ensuring that plaintext passwords are never persisted. The security configuration disables CSRF protection (appropriate for stateless REST API architectures) and enables CORS to allow HTTP requests from the separate frontend origin. Endpoint access control mandates that search and browse endpoints are publicly accessible (no authentication required), while watchlist management and personalized recommendation endpoints require an authenticated user session. This configuration provides a secure boundary between public and private application functionality.

Chapter 6
Experimental Setup

6. Experimental Setup

6.1 Dataset Details

The data source for CineRecommend is a publicly available movie metadata CSV dataset sourced from Kaggle. The dataset contains records for thousands of movies, each with associated fields including the movie title, a URL pointing to the poster image, a textual overview or synopsis, a pipe-delimited or comma-delimited string of genre tags, and a numeric popularity score derived from community interaction metrics.

The dataset is intentionally left in its raw, unprocessed form before being fed to the DataLoader to simulate real-world conditions. This includes deliberately retaining rows with missing titles, null popularity values, and malformed genre strings to validate the Self-Healing Pipeline's effectiveness.

6.2 Software and Hardware Setup

Component	Technology Used
Backend Language	Java 17
Backend Framework	Spring Boot
Security	Spring Security with BCrypt
Database	MongoDB (NoSQL)
Frontend	HTML5, CSS3, Vanilla JavaScript
Data Source	Static Kaggle CSV Dataset
Architecture Pattern	MVC (Model-View-Controller)

Table 3: Technology Stack

6.3 Database Schema Design

The MongoDB database, named stream_db, consists of two primary collections:

6.3.1 movies Collection

Each document in this collection represents a single movie entry. The schema includes the following fields:

- _id: ObjectId — MongoDB-generated unique identifier.

- title: String — The official movie title. Validated as non-null during ingestion.
- posterUrl: String — URL linking to the movie's poster image. May become invalid over time; handled by the frontend error recovery mechanism.
- overview: String — A textual description or synopsis of the movie's plot.
- genres: Array<String> — An array of genre tags (e.g., ['Action', 'Thriller']). Supports multi-genre classification without schema changes, a key advantage of NoSQL over relational databases.
- popularity: Double — A numeric score representing the movie's popularity. Default value of 0.0 assigned during ingestion if the raw value is malformed.

6.3.2 users Collection

Each document in this collection represents a registered user account. The schema includes:

- _id: ObjectId — MongoDB-generated unique identifier.
- username: String — The user's chosen display name. Unique constraint enforced at the application layer.
- email: String — User's email address, used as the primary login credential.
- password: String — BCrypt-hashed password. The plaintext password is never stored.
- favoriteGenres: Array<String> — An array of genres selected during registration. Used as the input for Phase 1 Content-Based recommendations.
- watchlist: Array<ObjectId> — An array of Movie ObjectIds representing the user's saved watchlist. Used as the behavioral input for Phase 2 Collaborative Filtering.

6.4 Performance Evaluation Parameters

The system is evaluated against the following performance parameters:

- Query Response Time: The elapsed time from the receipt of an HTTP request by the Spring Boot backend to the transmission of the JSON response. Target: under 200 milliseconds for search and recommendation queries.
- DataLoader Resilience: The percentage of intentionally corrupt CSV rows successfully intercepted by the validation logic during the startup ingestion test. Target: 100% interception with zero server crashes.
- Frontend Error Recovery: The percentage of dynamically rendered movie cards with dead poster URLs that are correctly removed from the DOM by the JavaScript error handler. Target: 100% recovery with no broken image icons visible to the user.

- Cold Start Resolution: Whether a newly registered user with zero watchlist entries receives a non-empty, genre-relevant list of recommendations on their first login. Target: immediate recommendations on first session.

Chapter 7
Results and Discussion

7. Results and Discussion

7.1 Recommendation Engine Performance

The Hybrid Recommendation Algorithm was tested across two distinct user scenarios. For a newly registered user who selected genres 'Action', 'Thriller', and 'Science Fiction' during signup and had an empty watchlist, the system immediately returned a ranked list of the top-rated movies within those genres. The Cold Start problem was fully resolved: the user received a meaningful, personalized recommendation list on their very first login session, without requiring any prior behavioral data.

For a returning user with a populated watchlist, the system transitioned seamlessly to Collaborative Filtering mode. By cross-referencing the user's watchlist against those of other registered users, the algorithm successfully identified peer users with similar taste profiles and recommended movies from their lists that the active user had not yet added. This produced serendipitous suggestions outside the user's explicit genre preferences, demonstrating the system's ability to introduce content discovery.

7.2 DataLoader — Self-Healing Validation Results

The DataLoader was stress-tested by intentionally introducing 50 malformed rows into the test CSV dataset. The corrupted entries included rows with null title fields, rows where the popularity column contained the string 'N/A' rather than a valid floating-point number, and rows with completely empty genre fields. The DataLoader successfully intercepted all 50 malformed records during the server startup initialization phase. In each case, the system either assigned the appropriate default fallback values or safely discarded the row, and the server completed its startup sequence without any exceptions or crashes. Database integrity was maintained at 100%, with only clean, well-formed documents persisted to the MongoDB movies collection.

7.3 Frontend Error Recovery Results

The fault-tolerant frontend was tested by deliberately replacing valid posterUrl values with expired or non-existent URLs in the movie documents. When the JavaScript rendered the movie grid and the browser's image request received a 404 response, the onerror event fired

correctly for every affected image element. The corresponding movie card container was removed from the DOM in each case, with no broken image icons remaining visible in the user interface. The remaining valid movie cards continued to render correctly without any layout disruption. The error recovery rate was 100% across all test cases.

7.4 Query Response Time

Search queries using MongoDB Regex-based partial title matching and genre-filtered recommendation queries were both consistently returned in under 200 milliseconds under normal testing conditions. This performance is attributable to MongoDB's native indexing on the _id field and the efficient use of Spring Data's MongoRepository abstraction, which generates optimized queries without requiring manual query construction. The sub-200ms response time ensures a seamless, responsive user experience that meets modern web application performance standards.

7.5 Discussion

The results collectively validate the core architectural thesis of CineRecommend: that fault tolerance can be embedded at every layer of a web application — the data ingestion layer, the algorithm layer, and the presentation layer — without sacrificing performance or requiring specialized hardware. The system achieves all six objectives defined in Chapter 4 and demonstrates that the engineering principles of SDG 9 can be applied practically within the scope of an academic software engineering project.

Chapter 8
Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

CineRecommend successfully achieves its core objective of delivering a highly resilient, personalized, and fault-tolerant movie recommendation web application. By moving away from single-algorithm models and adopting a context-sensitive Hybrid approach, the system effectively neutralizes the Cold Start problem for new users while enabling content discovery for returning users. The two-phase algorithm — Content-Based for empty watchlists and Collaborative for populated ones — proves that a lightweight, hardware-friendly recommendation engine can provide the benefits of both major paradigms without their respective drawbacks.

The integration of a Self-Healing DataLoader demonstrates that data infrastructure resilience can and should be engineered proactively into web applications, rather than treated as an operational concern to be addressed after a production crash. By validating and sanitizing every record during the startup ingestion phase, CineRecommend provides a structural guarantee of database integrity that prevents an entire class of runtime exceptions.

The dynamic frontend error recovery further reinforces the system's commitment to a consistently professional user experience under real-world network conditions, where media URLs routinely expire or become unavailable.

Collectively, these three engineering innovations — the hybrid algorithm, the self-healing pipeline, and the fault-tolerant frontend — embody the principles of SDG 9 (Industry, Innovation, and Infrastructure) by proving that resilient, innovative software systems can be designed and deployed accessibly on standard web infrastructure.

8.2 Future Scope

While the current system operates efficiently within its defined scope, several significant enhancements are identified for future development iterations:

- Live API Integration: The static CSV ingestion pipeline can be replaced with a live connection to the TMDB (The Movie Database) public API, enabling the system to automatically fetch metadata for newly released movies and maintain a continuously up-to-date database without manual data refreshes.

- Database Indexing and Pagination: To scale the application for millions of movie records and thousands of concurrent users, compound MongoDB indexes on the title, genres, and popularity fields should be implemented, combined with server-side pagination for all list endpoints, to maintain sub-100ms query performance at scale.
- Advanced Recommendation Metrics: The Collaborative Filtering phase can be enhanced with formal similarity metrics such as Pearson Correlation Coefficient or Cosine Similarity calculated over rating vectors, replacing the current intersection-based heuristic for more mathematically rigorous peer identification.
- User Ratings Module: Introducing a 1-to-5 star rating system for watched movies would provide richer behavioral signals for the Collaborative Filtering algorithm, enabling it to differentiate between movies a user watched and enjoyed versus those they watched and disliked.
- Deployment and Containerization: The Spring Boot application can be containerized using Docker and deployed to a cloud platform (such as AWS EC2 or Google Cloud Run), with a CI/CD pipeline for automated testing and deployment, taking the system from an academic prototype to a production-ready application.

References

[1] G. Singh, R. Kumar, and A. Sharma, "A Review of Movie Recommendation System: Limitations, Survey and Challenges," Electronic Letters on Computer Vision and Image Analysis (ELCVIA), vol. 19, no. 3, pp. 18–37, 2020.

[2] K. Jena, S. Mishra, and P. Das, "Neural Model-Based Collaborative Filtering for Recommendation Systems," IEEE Access, vol. 9, pp. 112401–112415, 2021.

[3] N. Bhalse and E. N. Ganesh, "Similarity Based Deep Learning Model for Movie Recommendation," E3S Web of Conferences, vol. 391, Art. no. 01143, 2023.

[4] M. Sankaran, "SVD-Based Matrix Factorization for Scalable Movie Recommendation Systems," International Journal of Advanced Computer Science and Applications (IJACSA), vol. 14, no. 6, 2023.

[5] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, "Recommender Systems Survey," Knowledge-Based Systems, vol. 46, pp. 109–132, Jul. 2013.

[6] A. Mourya, "A Survey on Movie Recommendation System Approaches and Challenges," Medium Research Publication, Mar. 2023.

[7] C. C. Aggarwal, Recommender Systems: The Textbook. New York, NY, USA: Springer, 2016.

[8] Spring Boot Documentation, "Spring Security Reference Guide," VMware, Inc., 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/